Builder

Builder · Build immutable HTTP requests with many optionals · Build with Waqar

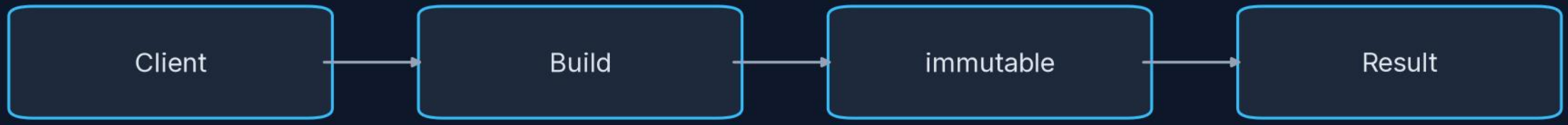
Today's Hook

- Problem: Build immutable HTTP requests with many optionals
- Pattern: Builder — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Build immutable HTTP requests with many optionals
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Builder: Build immutable HTTP requests with many optionals

Real-World Scenario

- Running example: Build immutable HTTP requests with many optionals
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

import com.buildwithwaqar.patterns.gof.creational.builder.HttpRequest;

/** Problem: construct API requests with many optional headers without telescoping ctors. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Builder problem: HTTP request with optionals ===");
        var req = HttpRequest.builder("https://api.example/v1/videos")
                .method("POST")
                .header("Authorization", "Bearer t")
                .body("{\"title\":\"Adapter\"}")
                .timeoutMs(3000)
                .build();
        System.out.println("method=" + req.method() + " auth=" + req.headers().get("Authorization")
                + " timeout=" + req.timeoutMs());
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;

/** Immutable HTTP request built with a fluent classic builder. */
public final class HttpRequest {
    private final String method;
    private final String url;
    private final Map<String, String> headers;
    private final String body;
    private final int timeoutMs;

    private HttpRequest(Builder b) {
        this.method = b.method;
        this.url = b.url;
        this.headers = Map.copyOf(b.headers);
        this.body = b.body;
        this.timeoutMs = b.timeoutMs;
    }
```

Solution (2)

```
    public String method() { return method; }
    public String url() { return url; }
    public Map<String, String> headers() { return headers; }
    public String body() { return body; }
    public int timeoutMs() { return timeoutMs; }

    public static Builder builder(String url) { return new Builder(url); }

    /** Modern: records + withers-style nested builder still works well. */
    public static final class Builder {
        private String method = "GET";
        private final String url;
        private final Map<String, String> headers = new LinkedHashMap<>();
        private String body = "";
        private int timeoutMs = 5_000;

        private Builder(String url) {
            this.url = Objects.requireNonNull(url);
        }

        public Builder method(String method) { this.method = method; return this; }
        public Builder header(String k, String v) { headers.put(k, v); return this; }
        public Builder body(String body) { this.body = body; return this; }
        public Builder timeoutMs(int timeoutMs) { this.timeoutMs = timeoutMs; return this; }
```

Solution (3)

```
        public HttpRequest build() {
            if (timeoutMs <= 0) throw new IllegalArgumentException("timeoutMs must be > 0");
            return new HttpRequest(this);
        }
    }
}

/** Java 25-friendly: compact record with smart constructor + defaults via static factory. */
record ApiCall(String method, String url, String body, int timeoutMs) {
    ApiCall {
        Objects.requireNonNull(method);
        Objects.requireNonNull(url);
        if (timeoutMs <= 0) throw new IllegalArgumentException("timeout");
        body = body == null ? "" : body;
    }

    static ApiCall get(String url) { return new ApiCall("GET", url, "", 5_000); }
    ApiCall withBody(String body) { return new ApiCall(method, url, body, timeoutMs); }
    ApiCall withTimeout(int ms) {
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.creational.builder;

import com.buildwithwaqar.patterns.gof.creational.builder.HttpRequest;

/** Problem: construct API requests with many optional headers without telescoping ctors. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Builder problem: HTTP request with optionals ===");
        var req = HttpRequest.builder("https://api.example/v1/videos")
                .method("POST")
                .header("Authorization", "Bearer t")
                .body("{\"title\":\"Adapter\"}")
                .timeoutMs(3000)
                .build();
        System.out.println("method=" + req.method() + " auth=" + req.headers().get("Authorization")
                + " timeout=" + req.timeoutMs());
    }
}
```

Walkthrough Output

```
=== Builder problem: HTTP request with optionals ===
method=POST auth=Bearer t timeout=3000
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Builder.

Related Patterns

- Contrast Builder with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Builder → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.